

PROJECT REPORT

FitBuddy – AI Fitness Plan Generator

AI-Powered Personalized Fitness Planning using Gemini Models

Submitted By	HARINI B
Register Number	212505066
Project Type	AI / Web Application
Technology	FastAPI, Google Gemini, Jinja2, SQLite, SQLAlchemy

A project report prepared based on the supplied FitBuddy sample project document.

CERTIFICATE

This is to certify that **HARINI B**, Register Number **212505066**, has prepared the project report entitled **“FitBuddy – AI Fitness Plan Generator”**. The project presents an AI-assisted web application for generating personalized fitness plans using user-provided goals and workout intensity.

Project Guide	Signature	Date
_____	_____	_____
Faculty / Mentor		

Student	Department / Institution
HARINI B	_____
212505066	_____

DECLARATION

I hereby declare that the project report titled **“FitBuddy – AI Fitness Plan Generator”** is prepared for academic project work. The report describes the proposed application, its architecture, development workflow, modules, testing approach, and deployment process based on the supplied project reference.

The project is designed to demonstrate how generative AI can be integrated with a web application to provide personalized workout planning, nutrition/recovery tips, and feedback-based refinement.

ABSTRACT

FitBuddy is a web-based AI fitness plan generator designed to create personalized workout plans from basic user information such as name, age, weight, fitness goal, and workout intensity. The application uses FastAPI for backend processing, Google Gemini models for generative AI, Jinja2 templates for the web interface, and SQLite with SQLAlchemy for data storage.

The system generates a structured seven-day workout plan containing warm-up, main workout, cooldown, and recovery guidance. A Gemini Flash based feature can provide concise nutrition or recovery tips. Users can also submit feedback such as requesting more cardio or additional rest days; the system sends the original plan and feedback to Gemini Pro to produce a revised plan.

The project demonstrates an end-to-end AI application workflow: user input, backend processing, AI generation, database storage, dynamic HTML rendering, feedback-driven refinement, and administrative viewing of user records and plans.

Keywords: Artificial Intelligence, Gemini, FastAPI, Fitness Plan, Jinja2, SQLite, SQLAlchemy, Personalized Workout

TABLE OF CONTENTS

No.	Section
1	Introduction
2	Problem Statement and Objectives
3	Project Scenarios
4	Technology and Prerequisites
5	System Architecture
6	Model Selection
7	Development Milestones
8	Core Functional Modules
9	Frontend Design
10	Database Design
11	FastAPI Routes and Workflow
12	Testing and Validation
13	Deployment
14	Project Requirements and Submission
15	Conclusion and Future Scope

1. INTRODUCTION

FitBuddy – AI Fitness Plan Generator is a web application that combines a simple user interface with generative AI to create personalized fitness guidance. Instead of giving every user the same routine, the system accepts individual details and fitness preferences and uses those inputs to produce a goal-oriented seven-day plan.

The application is centered around three user-facing capabilities: generating a workout plan, providing a nutrition/recovery tip, and updating an existing plan from user feedback. An administrative view is also included so that stored users and their original and updated plans can be reviewed.

The supplied project reference describes FitBuddy as a FastAPI application integrated with Google Gemini models, Jinja2 templates, and SQLite/SQLAlchemy. This combination provides a lightweight architecture suitable for academic development and demonstration.

The project is intended to demonstrate practical integration of AI services into a complete web application rather than only showing an isolated AI prompt.

1.1 Purpose

- Provide personalized seven-day workout plans from user inputs.
- Generate concise nutrition or recovery tips related to the selected fitness goal.
- Allow users to refine a generated plan through natural-language feedback.
- Store user information and generated plans for later viewing.
- Demonstrate an AI-enabled FastAPI web application workflow.

2. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

Many fitness applications provide fixed routines that may not directly match an individual's goal, intensity preference, or feedback. FitBuddy addresses this by using generative AI to produce a plan dynamically from user-provided information and to revise the plan when the user gives additional instructions.

Objectives

- Create a simple web interface for collecting fitness information.
- Generate a structured seven-day workout plan using Gemini.
- Include warm-up, main workout, cooldown, and recovery elements.
- Provide a nutrition/recovery tip based on the selected goal.
- Accept feedback and regenerate an improved version of the plan.
- Persist user and plan data using SQLite and SQLAlchemy.
- Provide an admin page for viewing stored users and plans.
- Run the application locally through FastAPI and Uvicorn.

Expected Outcome

The expected result is an interactive application in which a user enters fitness details, receives a customized plan, reviews supporting guidance, and can request changes through feedback.

3. PROJECT SCENARIOS

Scenario 1 – Personalized Plan

The user enters name, age, weight, fitness goal, and workout intensity. The application sends the information to the AI layer and returns a personalized seven-day workout plan.

Scenario 2 – Feedback and Revision

The user can submit feedback such as “more focus on cardio” or “include more rest days.” The original plan and feedback are passed to Gemini so that an updated plan can be generated.

Scenario 3 – Nutrition / Recovery Tip

The user can receive a concise practical tip related to the selected fitness goal. Gemini Flash is used in the reference architecture for this fast response.

Scenario 4 – Admin / Coach View

An administrator or coach can view stored user details and compare original and updated plans, helping demonstrate how the plan changes over time.

4. TECHNOLOGY AND PREREQUISITES

Technology	Purpose
Python	Application development and backend logic
FastAPI	Web framework and API routing
Google Gemini	Generative AI for workout plans and tips
Jinja2	Dynamic HTML template rendering
SQLite	Lightweight persistent database
SQLAlchemy	Database access and ORM layer
HTML / CSS	Frontend structure and styling
Uvicorn	Local ASGI application server
Git / GitHub	Source control and project submission

Prerequisites

- Python installed and available from the terminal.
- Gemini API access and an API key stored securely as an environment variable.
- Basic knowledge of FastAPI, HTML/CSS, Jinja2, and Python.
- SQLite/SQLAlchemy setup for local data storage.
- Git for maintaining and submitting the project repository.

Environment Setup

```
python -m venv fitbuddy-env  
# Windows  
fitbuddy-env\Scripts\activate
```

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

```
uvicorn app.main:app --reload
```

5. SYSTEM ARCHITECTURE

The system follows a layered architecture in which the frontend collects user data, FastAPI handles requests, Gemini provides AI-generated content, and SQLite/SQLAlchemy stores user and plan information.

Layer	Components	Responsibility
Frontend	HTML, CSS, Jinja2	Collect input and display results
Backend	FastAPI	Routes, validation, processing, responses
AI Layer	Gemini Pro / Flash	Workout generation, updates, tips
Database	SQLite + SQLAlchemy	Persist users and plans
Server	Uvicorn	Run FastAPI application locally

High-Level Data Flow

User → HTML Form → FastAPI Route → Gemini Model → Generated Plan → SQLite Storage → Jinja2 Result Page

Feedback Flow

Existing Plan + User Feedback → FastAPI → Gemini Pro → Updated Plan → Database Update → Result / Confirmation Page

6. MODEL SELECTION

The sample project describes experimentation with several language models before selecting Gemini for the final application. Models mentioned in the reference include Hugging Face T5, BART, DistilBERT, and Bloom.

Model	Use / Observation in Project Reference
Gemini 1.5 Pro	Selected for structured seven-day workout generation and plan updating
Gemini Flash	Selected for quick nutrition/recovery tips
T5	Experimented with during early model exploration
BART	Experimented with during early model exploration
DistilBERT	Experimented with during early model exploration
Bloom	Experimented with during early model exploration

Selection Considerations

- API-based integration fits naturally with FastAPI.
- Structured responses can be produced for day-wise fitness plans.
- Feedback can be passed together with the original plan to support revisions.
- Flash can be used where a short response is preferred.
- The architecture can separate plan generation from nutrition-tip generation.

7. DEVELOPMENT MILESTONES

Milestone 1 – Model Selection and Architecture

Select the AI models and define the FastAPI, database, template, and AI layers.

Milestone 2 – Core Functionality Development

Implement workout generation, nutrition/recovery tips, plan updates, and database functions.

Milestone 3 – routes.py Development

Create FastAPI endpoints for the home page, plan generation, feedback submission, and admin view.

Milestone 4 – Frontend Development

Build responsive Jinja2 pages for input, results, feedback, and administrative viewing.

Milestone 5 – Deployment

Prepare the environment, configure API credentials, run the application with Uvicorn, and verify the local deployment.

8. CORE FUNCTIONAL MODULES

Workout Generation

`generate_workout_gemini()` generates a seven-day plan containing warm-up, main workout, cooldown/recovery, and goal-oriented structure.

Nutrition / Recovery

`generate_nutrition_tip_with_flash()` uses Gemini Flash to produce a practical tip related to the user's fitness goal.

Workout Plan Update

`update_workout_plan()` sends the original plan and user feedback to Gemini Pro and returns a revised plan.

Database Operations

`database.py` provides functions such as `save_user()`, `save_plan()`, `update_plan()`, `get_original_plan()`, and `get_user()`.

Admin View

The `/view-all-users` route supports viewing stored users and their original/updated plans.

Example AI Prompt Structure

User profile:

Name: <name>

Age: <age>

Weight: <weight>

Goal: <goal>

Intensity: <intensity>

Task:

Generate a structured 7-day workout plan.

Include warm-up, main workout, cooldown/recovery,
and clear day-wise formatting.

9. FRONTEND DESIGN

The frontend is built using HTML, CSS, and Jinja2. The sample project describes a gym-themed visual design with bold typography, Google Fonts such as Roboto, Flexbox-based layout, rounded elements, hover effects, and responsive media queries.

Template	Purpose
index.html	Collects user information and fitness preferences
result.html	Displays the AI-generated plan and nutrition tip; accepts feedback
all_users.html	Displays stored users and original/updated plans for admin review

User Input Fields

- Name
- User ID
- Age
- Weight (kg)
- Fitness Goal
- Workout Intensity

Result Page

The result page displays the user's submitted details along with the generated workout plan and nutrition/recovery tip. A feedback form allows the user to request modifications to the current plan.

10. DATABASE DESIGN

SQLite is used as the local database and SQLAlchemy provides the ORM layer. The database stores user details and generated plan information so that the application can retrieve and update records.

Data	Description
User details	Name, user identifier, age, weight, goal, intensity
Original plan	The first AI-generated workout plan
Updated plan	The revised plan generated after feedback
Plan relationship	Associates plans with the corresponding user

Database Operations

- `save_user()` – stores user information.
- `save_plan()` – stores a generated plan.
- `update_plan()` – updates plan information after feedback.
- `get_original_plan()` – retrieves the original plan.
- `get_user()` – retrieves user information.

The database layer separates persistence from the AI generation layer, making the project easier to organize and maintain.

11. FASTAPI ROUTES AND WORKFLOW

Route	Method / Function	Purpose
/	GET	Show the main input page
/generate-workout	POST	Receive user input and generate the workout plan
/submit-feedback	POST	Receive feedback and generate an updated plan
/view-all-users	GET	Display stored users and plan information

Route Workflow

1. The user opens the home page. 2. The form collects personal and fitness information. 3. FastAPI receives the form submission. 4. The AI generation function creates the plan. 5. User information and plan data are stored. 6. Jinja2 renders the result page. 7. If feedback is submitted, the existing plan and feedback are sent to the update function. 8. The updated plan is stored and displayed.

FastAPI also provides interactive API documentation at the local **/docs** endpoint when the development server is running.

12. TESTING AND VALIDATION

Testing focuses on verifying that each core feature accepts expected inputs, produces a response, and maintains the user flow described in the project reference.

Test Case	Input / Action	Expected Result
TC01	Open /	Input form is displayed
TC02	Submit valid user details	Seven-day plan is generated
TC03	Select different intensity	Plan reflects the selected intensity
TC04	Request nutrition tip	Goal-related tip is displayed
TC05	Submit feedback	Updated plan is generated
TC06	Open /view-all-users	Stored user and plan records are shown
TC07	Run application locally	FastAPI server starts successfully
TC08	Open /docs	Interactive API documentation is available

Validation Points

- Check required form fields before processing.
- Confirm the Gemini API key is available through environment configuration.
- Verify that generated content is stored against the correct user.
- Check that feedback updates the intended plan.
- Confirm the frontend renders AI output clearly.

13. DEPLOYMENT

The sample project focuses on local deployment using FastAPI and Uvicorn. The environment isolates project dependencies and keeps the Gemini API key outside the source code.

Environment Variable

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Run the Application

```
uvicorn main:app --reload
# or, depending on project structure:
uvicorn app.main:app --reload
```

Local Access

The application can be opened through the local server address, commonly **http://127.0.0.1:8000**. FastAPI documentation is available through **http://127.0.0.1:8000/docs** when the server is running.

Deployment Checklist

- Create and activate the virtual environment.
- Install the required Python packages.
- Configure the Gemini API key securely.
- Confirm the database is initialized.
- Start Uvicorn.
- Test the home, generation, feedback, and admin routes.
- Verify that the frontend templates render correctly.

14. PROJECT REQUIREMENTS AND SUBMISSION

The supplied reference describes an academic submission structure with a public GitHub repository, phase-wise project organization, and a demonstration video.

Recommended Phase Structure

Phase 1 – Brainstorming & Ideation

Phase 2 – Requirement Analysis

Phase 3 – Project Design

Phase 4 – Project Planning

Phase 5 – Project Development

Phase 6 – Project Testing

Phase 7 – Project Documentation

Phase 8 – Project Demonstration

Repository and Demo

- Maintain the source code and project documents in a public GitHub repository as required by the reference.
- Keep the project files organized by development phase.
- Prepare a demonstration video with screen sharing and voice-over.
- Explain the project name, purpose, benefits, execution/working process, and final output during the demonstration.

The reference notes a team size of 2–5, with 4 described as the standard team size. This report is prepared with the supplied student details for HARINI B.

15. CONCLUSION AND FUTURE SCOPE

FitBuddy – AI Fitness Plan Generator demonstrates how generative AI can be integrated into a practical web application for personalized fitness planning. The project combines FastAPI for backend routing, Google Gemini for AI generation, Jinja2 for dynamic web pages, and SQLite with SQLAlchemy for persistent storage.

The system supports a complete interaction cycle: entering fitness details, generating a seven-day workout plan, receiving a nutrition/recovery tip, submitting feedback, and obtaining an updated plan. The administrative page further demonstrates how stored user and plan information can be reviewed.

Future Scope

- Add authentication for users and administrators.
- Add progress tracking and historical plan comparison.
- Provide richer exercise media and descriptions.
- Support additional fitness goals and activity preferences.
- Add structured meal-planning features.
- Improve validation and safety guidance for generated fitness content.
- Deploy the application to a cloud platform after local validation.
- Add analytics to summarize user progress and plan changes.

Final Summary

FitBuddy provides a clear example of an AI-enabled fitness web application in which user feedback can influence subsequent generated content. Its modular design makes it suitable for further development, testing, and deployment.

APPENDIX A – PROJECT FILE STRUCTURE

```
FitBuddy/
├──
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── database.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   └── updated_plan.py
├──
├── templates/
│   ├── index.html
│   ├── result.html
│   └── all_users.html
├──
├── static/
│   └── css / images
├──
├── requirements.txt
├── .env
└── README.md
```

APPENDIX B – KEY FUNCTIONS

Function	Module	Purpose
generate_workout_gemini()	gemini_generator.py	Generate seven-day workout plan
generate_nutrition_tip_with_flash()	gemini_flash_generator.py	Generate nutrition/recovery tip
update_workout_plan()	updated_plan.py	Revise plan from user feedback
save_user()	database.py	Save user information
save_plan()	database.py	Save generated plan
update_plan()	database.py	Update stored plan
get_original_plan()	database.py	Retrieve original plan
get_user()	database.py	Retrieve user information

END OF PROJECT REPORT